

•



- [Feature Engineering](#)
- [Using PQL in Features](#)
- Function Calls in PQL

On this page

Function Calls in PQL

Was this page helpful? 

Function calls allow you to execute predefined functions on current values. There are two types of functions that can be called:

- [Built-in functions](#) of Pulse
- [User-Defined Functions \(UDFs\)](#) â a set of additional functions bundled with Pulse

This topic gives an overview of the possible function calls, and points you to the right topics to learn more.

Built-in functions

Checking NULLs

Return type	Method	Description
bool	<code>isNull(int? a)</code>	Checks if an integer value is null.
bool	<code>isNull(long? a)</code>	Checks if a long value is null.
bool	<code>isNull(double? a)</code>	Checks if a double value is null.
bool	<code>isNull(float? a)</code>	Checks if a float value is null.
bool	<code>isNull(bool? a)</code>	Checks if a boolean value is null.
bool	<code>isNull(string? a)</code>	Checks if a string value is null.
bool	<code>isNull(tuple? a)</code>	Checks if a tuple value is null.

Math functions

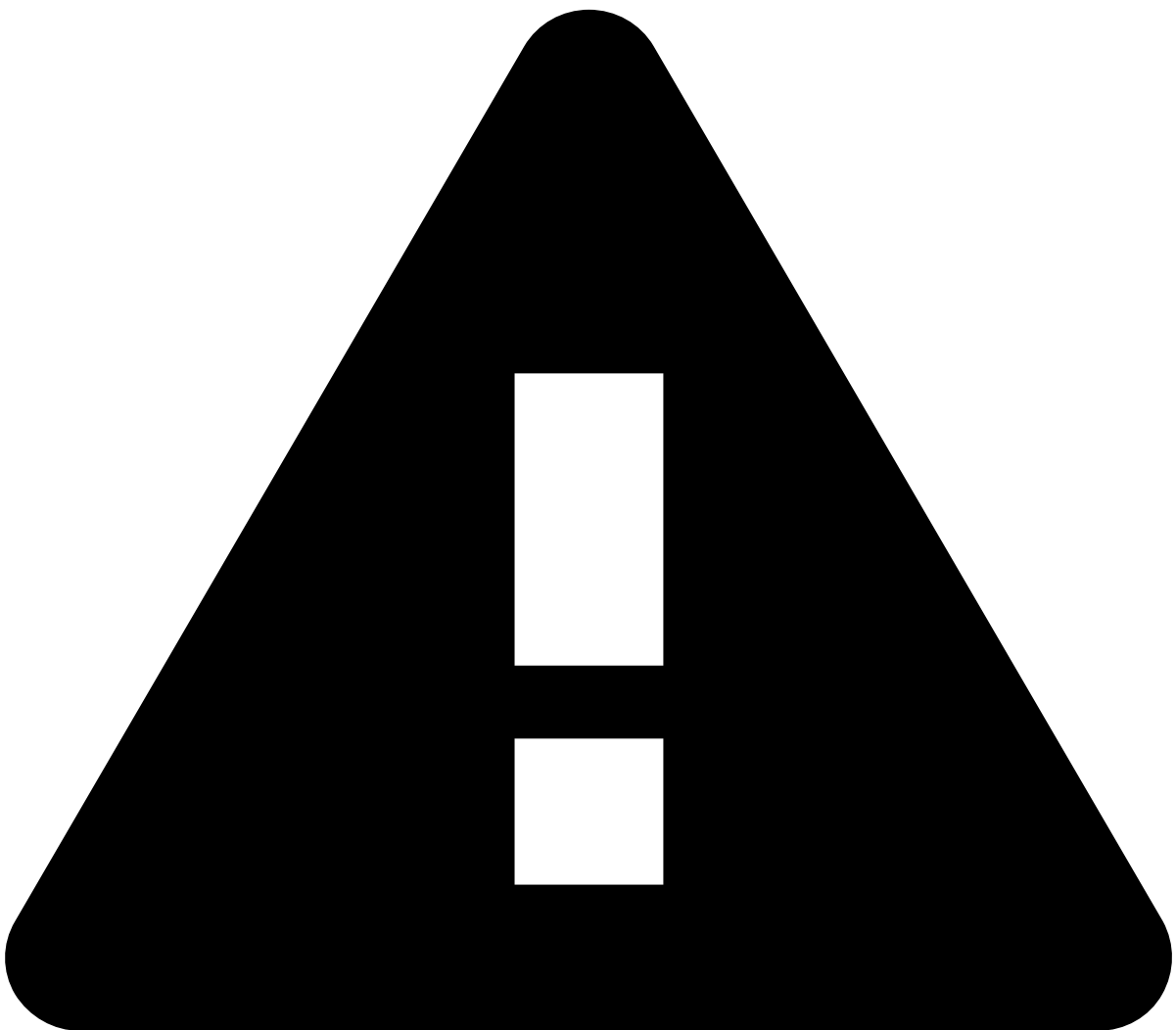
All methods of the [java.lang.Math](#) package are available in PQL. Besides those methods, you can use the following math functions:

Return type	Method	Description
int	<code>max(list_of_int)</code>	Returns the maximum value of a list of numbers.
long	<code>max(list_of_long)</code>	Returns the maximum value of a list of numbers.
double	<code>max(list_of_double)</code>	Returns the maximum value of a list of numbers.
float	<code>max(list_of_float)</code>	Returns the maximum value of a list of numbers.
int	<code>min(list_of_int)</code>	Returns the minimum value of a list of numbers.
long	<code>min(list_of_long)</code>	Returns the minimum value of a list of numbers.
double	<code>min(list_of_double)</code>	Returns the minimum value of a list of numbers.
float	<code>min(list_of_float)</code>	Returns the minimum value of a list of numbers.
boolean	<code>between(int val, int left, int right)</code>	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.
boolean	<code>between(long val, long left, long right)</code>	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.
boolean	<code>between(double val, double left, double right)</code>	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.
boolean	<code>between(float val, float left, float right)</code>	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.

Strings

Return type	Method	Description
string	<code>str(int a)</code>	Returns a new string that is the textual representation of <code>a</code> .
string	<code>str(long a)</code>	Returns a new string that is the textual representation of <code>a</code> .
string	<code>str(double a)</code>	Returns a new string that is the textual representation of <code>a</code> .
string	<code>str(float a)</code>	Returns a new string that is the textual representation of <code>a</code> .
string	<code>str(bool a)</code>	Returns a new string that is the textual representation of <code>a</code> .
string	<code>str(string a)</code>	Returns a new string that is the textual representation of <code>a</code> .

Return type	Method	Description
int	indexOf(string s, string c)	Returns the index within string <code>s</code> of the first occurrence of the string <code>c</code> .
string	substring(string s, int begin)	Returns a new string that is a substring of string <code>s</code> .
string	substring(string s, int begin, int end)	Returns a new string that is a substring of string <code>s</code> .
bool	contains(string s, string c)	Returns <code>true</code> if string <code>c</code> is a substring of string <code>s</code> , or <code>false</code> otherwise.
bool	startsWith(string s, string c)	Returns <code>true</code> if string <code>c</code> is a prefix of string <code>s</code> , or <code>false</code> otherwise.
bool	endsWith(string s, string c)	Returns <code>true</code> if string <code>c</code> is a suffix of string <code>s</code> , or <code>false</code> otherwise.



Container functions in `Map` and `Metrics` operators

Functions that operate on collections (like `List` s, `Set` s or `Map` s) are not supported in the PQL fields of [Map Operators](#) or [Metric Operators](#).

For example, using `contains` or `indexOf` on collections in PQL fields of such operators will result in an error.

The recommended alternative is to use [UDFs \(User Defined Functions\)](#) instead of these built-in functions.

UDF calls

Pulse comes with a set of [User-Defined Functions \(UDFs\)](#) that can be used in addition to the built-in functions.

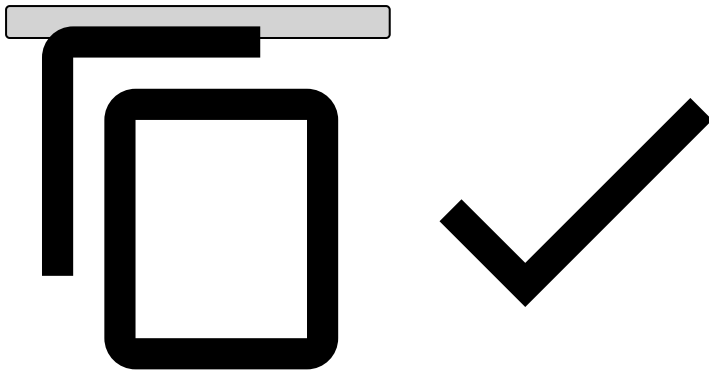


UDFs in Metrics

When using UDFs in metric filters and aggregations, UDFs receive fields from a single event and not from a collection of events.

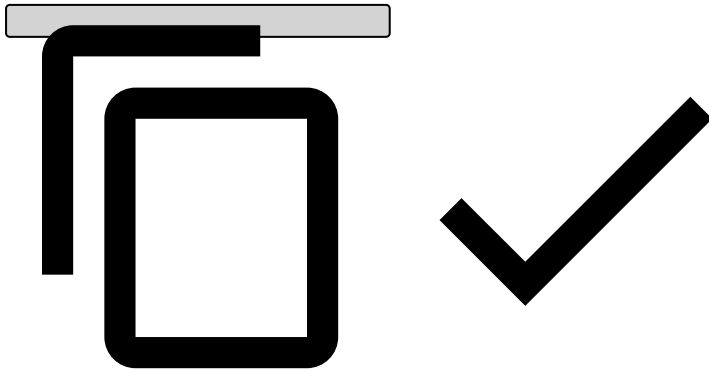
You can use the following syntax to call a UDF:

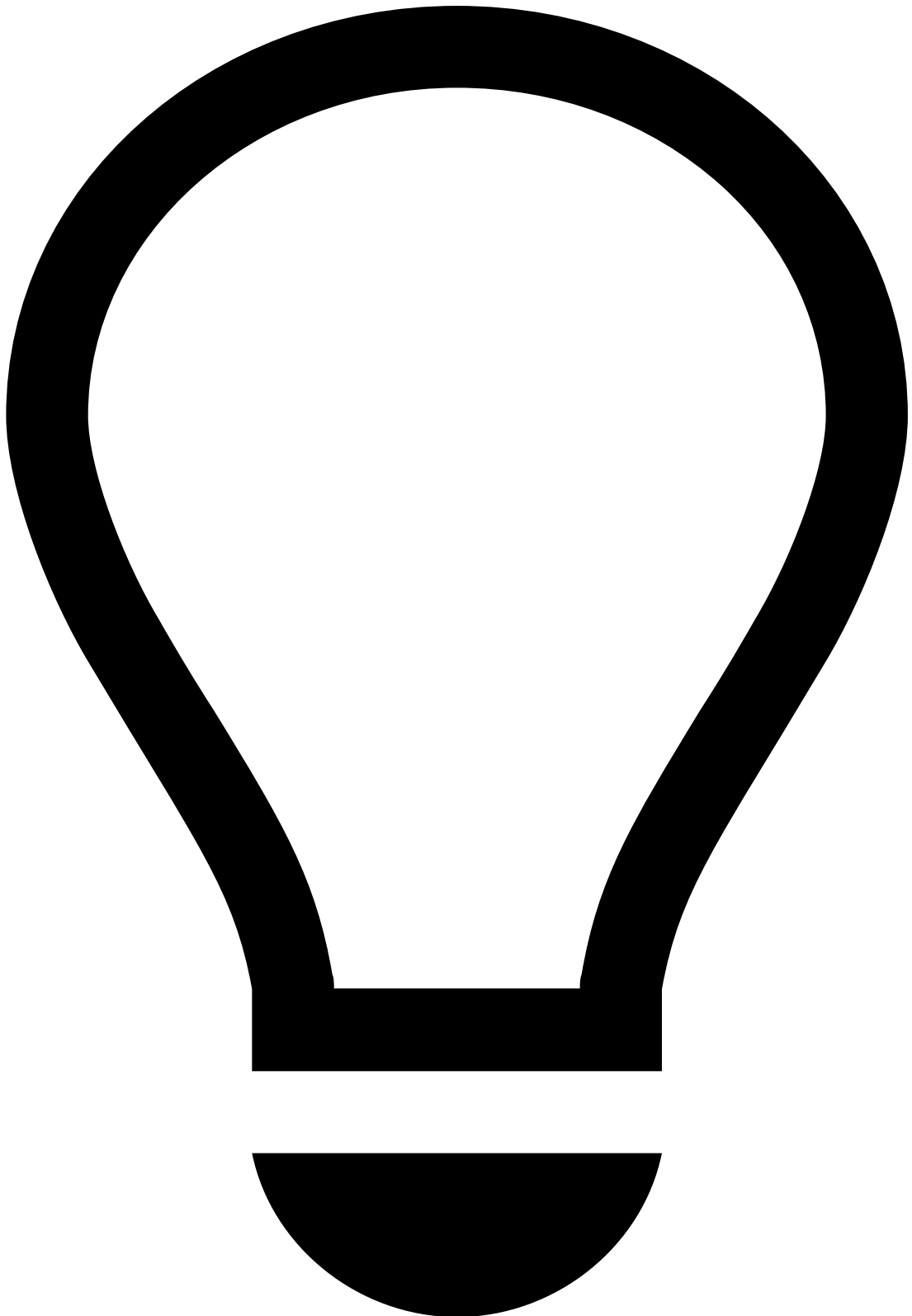
```
<class_name>.<function_name> ( <arguments> )
```



For example, to concatenate two strings:

```
Strings.strcat("One", "Two") // Returns "OneTwo"
```





java regex syntax

Pulse uses the Java regex flavor for UDFs, which means that all regular expressions must conform to the syntax and rules of Java's `java.util.regex.Pattern` class.

According to the usual regex syntax, when you need to match a literal special character that has meaning in regex (e.g. `*`, `+`, `?`, `.`), you must escape it using a backslash (`\`) within the regex pattern (e.g. `\*`, `\+`, and so on).

However, because the regex pattern itself is often passed as a Java String literal, the backslash (`\`) used for escaping the regex special character must itself be escaped according to Java String rules.

Therefore, to include an escaped character like `\*` (match literal asterisk) in a regular expression defined as a Java string, you must write it as `\\*`.